uniandes.dpoo.boletamaster /SRC
Cliente
AbstractClass
Usuario
-login: String
-password: String
-nombre: String
-saldo: double
+ Usuario(login:String,password:String,nombre:String,saldo:double)
+ autenticar(login:String,password:String):boolean
+ consultarSaldo():double
+ acreditarSaldo(monto:double):void
+ usarSaldo(monto:double):void
+ getLogin():String
+ setLogin(login:String):void
+ getPassword():String
+ setPassword(password:String):void
+ getNombre():String
+ setNombre(nombre:String):void
+ getSaldo():double
+ setSaldo(saldo:double):void
Administrador
-idAdministrador:String
+ Administrador(ganancias:double,idAdministrador:String,venuesAprobados:ArrayList<Venue>,
login:String,password:String,nombre:String,saldo:double)
+ aprobarVenue(venue:Venue,decision:boolean):void
+ cancelarEvento(evento:Evento):void
+ decidirReembolso(solicitud:String,decision:boolean):boolean
+ fijarCargoEmision(cargo:double,tiquete:Tiquete):void
+ fijarCargoServicio(tipoEvento:TipoEvento,porcentaje:double):void
+ getCargoEmision():double
+ getCargoServicio(tipoEvento:TipoEvento):double
+ getGanancias():double
+ getIdAdministrador():String
+ getIdAdministrador():String
+ setIdAdministrador(idAdministrador:String):void
+ getVenuesAprobados():ArrayList<Venue>
+ setVenuesAprobados(venuesAprobados:ArrayList<Venue>):void
+ setGanancias(ganancias:double):void
+ verReporteGanancias(login:String,password:String):doubleuble
Cliente
-idCliente:String
+ Cliente(login:String,password:String,nombre:String,saldo:double,idCliente:String)
+ getIdCliente():String
+ setIdCliente(idCliente:String):void
+ getTiquetes():ArrayList<Tiquete>
+ setTiquetes(tiquetesNuevos:ArrayList<Tiquete>):void
+ agregarTiquete(tiquete:Tiquete):void
+ eliminarTiquete(tiquete:Tiquete):boolean
+ transferirTiquete(receptor:Cliente,idTiquete:int,password:String):boolean
organizador
-idOrganizador: int
+ getCortesias(): List<Tiquete>
+ agregarCortesia(t: Tiquete): boolean
+ getFinanzas(): double
+ setFinanzas(finanzas: double): void
+ getIdOrganizador(): String
+ setIdOrganizador(idOrganizador: String): void
+ getEventos(): Evento[]
+ crearEvento(administrador: Administrador,idEvento: String,nombre: String,
fecha: LocalDate,hora: LocalTime,estado: String,tipoEvento: TipoEvento,
venue: Venue,oferta: Oferta,organizador: Organizador,
tiquetes: ArrayList<Tiquete>): Evento
+ asignarLocalidades(venue: Venue,evento: Evento,nombre: String,precio: double,
numerada: boolean,cupos: int,localidades: ArrayList<Localidad>): void
+ crearOferta(idLocalidad: String,nombre: String,precioBase: double,
numerada: boolean,localidad: Localidad,evento: Evento): Oferta
+ compraComoCortesia(transaccion: Transaccion,tiquete: Tiquete): Transaccion
+ consultarFinanzas(login: String,password: String): double
+ registrarEvento(e: Evento): void
comprador
MarketPlace
contraOferta
-comprador: Cliente
-estado: EstadoContraOferta
-fechaCreacion: LocalDateTime
-id: String
-monto: double
+ContraOferta(String, Cliente, double)
+ContraOferta(String, Cliente, double, EstadoContraOferta, LocalDateTime)
+getComprador(): Cliente
+getEstado(): EstadosContraOferta
+getFechaCreacion(): LocalDateTime
+getId(): String
+getMonto(): double
setEstado(EstadoContraOferta): void
estadoContraOferta
«Enum»
ACEPTADA
PENDIENTE
RECHAZADA
estadoOferta
«Enum»
ACTIVA
CANCELADA_ADMIN
CANCELADA_VENDEDOR
VENDIDA
ofertaMarketPlace
-contraofertas: List<ContraOferta>
-estado: EstadoOferta
-fechaCreacion: LocalDateTime
-id: String
-precioInicial: double
-tiquetes: List<Tiquete>
-vendedor: Cliente
OfertaMarketPlace(String, Cliente, List<Tiquete>, double, EstadoOferta, List<ContraOferta>,
LocalDateTime) : Constructor
getContraofertas(): List<ContraOferta>
getEstado(): EstadoOferta
getFechaCreacion(): LocalDateTime
getId(): String
getPrecioInicial(): double
getTiquetes(): List<Tiquete>
getVendedor(): Cliente
permiteAvendedor(Cliente): boolean
setEstado(EstadoOferta): void
VenuesAprobados
eventos
tiquetes
consola
clienteApp
ClienteApp() : void
+aceptarContraOferta(Scanner, BoletaMasterSystem, Cliente) : void
+autenticar(Scanner, BoletaMasterSystem, Cliente) : void
+cancelarOfertaS(Scanner, BoletaMasterSystem, Cliente) : void
+comprarTiquetes(BoletaMasterSystem, Cliente) : void
+listarTiquetes(Cliente) : void
+main(String[]) : void
+mostrarContraOfertasPendientes(BoletaMasterSystem, Cliente) : void
+mostrarOfertasActivas(BoletaMasterSystem, Cliente) : void
+mostrarOfertasDeVendedor(BoletaMasterSystem, Cliente) : void
+rechazarContraOferta(Scanner, BoletaMasterSystem, Cliente) : void
aorganizadorApp
+OrganizadorApp() : void
+autenticar(Scanner, BoletaMasterSystem, Organizador) : void
+listarEventos(BoletaMasterSystem, Organizador) : void
+mostrarFinanzas(Organizador, String, String) : void
adminApp
+autenticar(Scanner, BoletaMasterSystem, Administrador) : void
+cancelarOfertaS(Scanner, BoletaMasterSystem, Cliente) : void
+listarOfertas(BoletaMasterSystem) : void
+mostrarLog(BoletaMasterSystem, Administrador, String, String) : void
+mostrarMenu() : void
+AdminApp() : void
tiquetes
Transaccion
-idTransaccion: int
-fecha: Date
-monto: double
+ Transaccion(idTransaccion:int,fecha:LocalDateTime,
monto:double,cliente:Cliente)
+ getClienteC():Cliente
+ setCliente(cliente:Cliente):void
+ getFecha():LocalDateTime
+ setFecha(fecha:LocalDateTime):void
+ getIdTransaccion():int
+ setIdTransaccion(id:int):void
+ getMonto():double
+ setMonto(monto:double):void
Emite
AbstractClass
Tiquete
- idTiquete: int
- precio: double
- cargoServicio: double
- cargoEmision: double
- estado: String
+ Tiquete(cliente:Cliente,idTiquete:int,precio:double,cargoServicio:double,
cargoEmision:double,estado:String,localidad:Localidad,evento:Evento)
+ calcularValorTotal():double
+ getCargoEmision():double
+ setCargoEmision(cargoEmision:double):void
+ getCargoServicio():double
+ setCargoServicio(cargoServicio:double):void
+ getCliente():Cliente
+ setCliente(cliente:Cliente):void
+ getEstado():String
+ setEstado(estado:String):void
+ getEvento():Evento
+ setEvento(evento:Evento):void
+ getIdTiquete():int
+ setIdTiquete(idTi):void
+ getLocalidad():Localidad
+ setLocalidad(localidad:Localidad):void
+ getPrecio():double
+ setPrecio(precio:double):void
TiqueteBasico
-numeroAsiento: int
-localidadNumerada : bool
+ TiqueteBasico(cliente:Cliente,idTiquete:int,precio:double,
cargoServicio:double,cargoEmision:double,estado:String,
localidad:Localidad,evento:Evento,numeroAsiento:Integer,
localidadNumerada:boolean)
+ asignarAsiento(numAsiento:int):void
+ getNumeroAsiento():Integer
+ setNumeroAsiento(numeroAsiento:Integer):void
+ isLocalidadNumerada():boolean
+ setLocalidadNumerada(localidadNumerada:boolean):void
tiqueteBasico
AbstractClass
PaqueteTiquetes
-tiquetes: List<tiquete>
-transferible:boolean
+ PaqueteTiquetes(transferible:boolean)
+ agregarTiquete(tiquete:Tiquete):void
+ esTransferible():boolean
+ getTiquetes():List<Tiquete>
+ limpiarTiquetes():void
+ setTransferible(transferible:boolean):void
TiqueteMultiple
-cantidadEntradas: int
-precioTotal: double
+ TiqueteMultiple(cantidadEntradas:int,precioTotal:double)
+ calcularPrecioUnitario():double
+ getCantidadEntradas():int
+ setCantidadEntradas(cantidadEntradas:int):void
+ getPrecioTotal():double
+ setPrecioTotal(precioTotal:double):void
TiqueteTemporada
-cantidadEventos: int
-precioTotal: double
+ TiqueteTemporada(cantidadEventos:int,precioTotal:double)
+ definirEventos(eventos:ArrayList<Evento>):void
+ getCantidadEventos():int
+ setCantidadEventos(cantidadEventos:int):void
+ getPrecioTotal():double
+ setPrecioTotal(precioTotal:double):void
+ getEventosTemporada():List<Evento>
PaqueteDeluxe
-beneficios: String
-tiquetesIncluidos: Tiquete
+ PaqueteDeluxe(beneficiosIniciales:Collection<String>,
tiquetesIncluidosIniciales:Collection<Tiquete>)
+ agregarBeneficio(b:String):void
+ agregarCortesia(t:Tiquete):void
+ agregarTiquete(tiquete:Tiquete):void
+ getBeneficios():List<String>
+ getCortesias():List<Tiquete>
+ todosLosAccesos():List<Tiquete>
eventos
evento
-idEvento: int
-nombre: String
-fecha: Date
-hora: Time
-estado: String (activo, cancelado, etc.)
+ Evento(administrador:Administrador,idEvento:String,nombre:String,
fecha:LocalDate,hora:LocalTime,estado:String,tipoEvento:TipoEvento,
venue:Venue,oferta:Oferta,organizador:Organizador,tiquetes:ArrayList<Tiquete>)
+ asociarVenue(venue:Venue):void
+ cancelar(motivo:String):void
+ getAdministrador():Administrador
+ setAdministrador(administrador:Administrador):void
+ getIdEvento():String
+ setIdEvento(idEvento:String):void
+ getNombre():String
+ setNombre(nombre:String):void
+ getFecha():LocalDate
+ setFecha(fecha:LocalDate):void
+ getHora():LocalTime
+ setHora(hora:LocalTime):void
+ getEstado():String
+ setEstado(estado:String):void
+ getTipoEvento():TipoEvento
+ setTipoEvento(tipoEvento:TipoEvento):void
+ getVenue():Venue
+ setVenue(venue:Venue):void
+ getOferta():COferta
+ setOferta(oferta:Oferta):void
+ getOrganizador():Organizador
+ setOrganizador(organizador:Organizador):void
+ getTiquetes():ArrayList<Tiquete>
+ setTiquetes(tiquetes:ArrayList<Tiquete>):void
+ registrarTiquete(tiquete:Tiquete):void
+ getVendidos():int
+ getCapacidadMaxima():int
+ getPorcentajeVenta():double
venue
Venue
-idVenue: int
-nombre: String
-ubicacion: String
-capacidad: int
-eventosProgramados: List<Evento>
+ Venue(idVenue:String,nombre:String,ubicacion:String,capacidadMaxima:int,
localidades:ArrayList<Localidad>)
+ agregarLocalidad(localidad:Localidad):void
+ registrarEvento(evento:Evento):void
+ removerEvento(evento:Evento):void
+ tieneEventoEnFecha(fecha:LocalDate):boolean
+ getIdVenue():String
+ setIdVenue(idVenue:String):void
+ getNombre():String
+ setNombre(nombre:String):void
+ getUbicacion():String
+ setUbicacion(ubicacion:String):void
+ getCapacidadMaxima():int
+ setCapacidadMaxima(capacidadMaxima:int):void
+ getLocalidades():ArrayList<Localidad>
+ setLocalidades(localidades:ArrayList<Localidad>):void
oferta
- idLocalidad: int
- nombre: String
- precioBase: double
- numerada: boolean
+ Oferta(localidad:Localidad,evento:Evento,porcentaje:double,
inicio:LocalDateTime,fin:LocalDateTime)
+ Oferta(idLocalidad:String,nombre:String,precioBase:double,
numerada:boolean,localidad:Localidad,evento:Evento)
+ activa(now:LocalDateTime):boolean
+ aplicar(precioBase:double,now:LocalDateTime):double
+ getLocalidad():Localidad
+ getEvento():Evento
+ getPorcentaje():double
+ getInicio():LocalDateTime
+ getFin():LocalDateTime
tipoEvento
«Enum»
conciertos
eventos culturales
religiosos
deportivos varios
Localidad
-nombre: String
-precioBase: double
-numerada: boolean
-numAsientos: int
+ Localidad(oferta:Oferta,tiquetes:ArrayList<Tiquete>,nombre:String,
precioBase:double,numerada:boolean,numeroAsientos:int)
+ agregarTiquete(tiquete:Tiquete):void
+ fijarPrecio(precio:double):void
+ getNombre():String
+ setNombre(nombre:String):void
+ getPrecioBase():double
+ setPrecioBase(precioBase:double):void
+ isNumerada():boolean
+ setNumerada(numerada:boolean):void
+ getNumeroAsientos():int
+ setNumeroAsientos(numeroAsientos:int):void
+ getTiquetes():ArrayList<Tiquete>
+ setTiquetes(tiquetes:ArrayList<Tiquete>):void
+ getOferta():COferta
+ setOferta(oferta:Oferta):void
localidades
manager / Persistencia
«Interface»
AppBase
autenticar(Scanner, String, String)
mostrarMenu()
gestionarOfertas()
mostrarLog(BoletaMasterSystem, String, String)
listarTiquetes()
aceptarContraOferta(Scanner, BoletaMasterSystem, Cliente)
rechazarContraOferta(Scanner, BoletaMasterSystem, Cliente)
accede
accede
marketplaceService
-logSistema: LogSistema
-state: BoletaMasterState
+MarketplaceService(BoletaMasterState): MarketplaceService
+aceptarContraOferta(Cliente, String, String): void
+buscarOferta(String): Optional<OfertaMarketPlace>
+cancelarOfertaPorAdministrador(Administrador, String): void
+cancelarOfertaPorVendedor(Cliente, String): void
+comprarOferta(Cliente, String): void
+contraOfertasPendientes(Cliente): Map<OfertaMarketPlace, List<ContraOferta>>
+crearContraOferta(Cliente, String, double): ContraOferta liberarTiquetes(OfertaMarketPlace): void
+obtenerOfertasActivas(): List<OfertaMarketPlace>
+obtenerOfertasPorVendedor(Cliente): List<OfertaMarketPlace>
+obtenerTodasLasOfertas(): List<OfertaMarketPlace>
+publicarOferta(Cliente, List<Integer>, double, String): OfertaMarketPlace
+rechazarContraOferta(Cliente, String, String): void
+transferirTiquetes(OfertaMarketPlace, Cliente): void
+validarOfertaDeVendedor(Cliente, String): OfertaMarketPlace
gestiona
gestiona
boletaMasterState
-administrador: Administrador
-clientesPorLogin: Map<String, Cliente>
-eventosPorId: Map<String, Evento>
-logSistema: LogSistema
-ofertasPorId: Map<String, OfertaMarketPlace>
-organizadoresPorLogin: Map<String, Organizador>
-paquetesRaw: JSONArray
-tiquetesEnOferta: Map<Integer, String>
-tiquetesPorId: Map<Integer, Tiquete>
+getAdministrador(): Administrador
+getClientesPorLogin(): Map<String, Cliente>
+getEventosPorId(): Map<String, Evento>
+getLogSistema(): LogSistema
+getOfertasPorId(): Map<String, OfertaMarketPlace>
+getOrganizadoresPorLogin(): Map<String, Organizador>
+getPaquetesRaw(): JSONArray
+getTiquetesEnOferta(): Map<Integer, String>
+getTiquetesPorId(): Map<Integer, Tiquete>
+reset(): void
+setAdministrador(Administrador): void
+setPaquetesRaw(JSONArray): void
boletaMasterSystem
-dataStore: JsonDataStore
-marketplaceService: MarketplaceService
-state: BoletaMasterState
+BoletaMasterSystem(jsonDataStore): BoletaMasterSystem
+BoletaMasterSystem(Path, Path, Path, Path): BoletaMasterSystem
+desdeDirectorio(String): BoletaMasterSystem
+aceptarContraOferta(Cliente, String): void
+autenticarAdministrador(String, String): Optional<Administrador>
+autenticarCliente(String, String): Optional<Cliente>
+autenticarOrganizador(String, String): Optional<Organizador>
+buscarOferta(String): Optional<OfertaMarketPlace>
+cancelarOfertaPorAdministrador(Administrador, String): void
+cancelarOfertaPorVendedor(Cliente, String): void
+cargarDatos(): void
+comprarOferta(Cliente, String): void
+contraOfertasPendientes(Cliente): Map<OfertaMarketPlace, List<ContraOferta>>
+crearContraOferta(Cliente, String, double): ContraOferta
+ejecutarYGuardar(Runnable): void
+ejecutarYGuardar(Supplier<T>) : T
+getAdministrador(): Administrador
+getEventosOrganizador(Organizador): Collection<Evento>
+getLogSistema(): LogSistema
+guardarDatos(): void
+marketplace(): MarketplaceService
+obtenerOfertasActivas(): List<OfertaMarketPlace>
+obtenerOfertasPorVendedor(Cliente): List<OfertaMarketPlace>
+obtenerTiquetesCliente(Cliente): List<Tiquete>
+publicarOferta(Cliente, List<Integer>, double, String): OfertaMarketPlace
+requireState(): BoletaMasterState
datosClientes
almacena
registro
mainPersistenciaDemo
main(String[]): void
jsonDataStore
eventosPath: Path
logPath: Path
ofertasPath: Path
paquetesPath: Path
tiquetesPath: Path
usuariosPath: Path
-fromDirectory(String): JsonDataStore
-buscarLocalidad(Venue, String): Localidad
-cargarEventos(BoletaMasterState): void
-cargarLog(BoletaMasterState): void
-cargarOfertas(BoletaMasterState): void
-cargarPaquetes(BoletaMasterState): void
-cargarTiquetes(BoletaMasterState): void
-cargarUsuarios(BoletaMasterState): void
-guardarLog(BoletaMasterState): void
-guardarOfertas(BoletaMasterState): void
-guardarPaquetes(BoletaMasterState): void
-guardarTiquetes(BoletaMasterState): void
-guardarUsuarios(BoletaMasterState): void
-load(): BoletaMasterState
-parseDateTime(String): LocalDateTime
-save(BoletaMasterState): void
JsonFiles «utility»
+ensureDir(path:Path) : void
+write(file:Path, content:String) : void
+read(file:Path) : String
PersistenciaVenuesJson
-ID_VENUE : String
-NOMBRE : String
-UBICACION : String
-CAPACIDAD : String
-LOCALIDADES_IDS : String
-ID_LOCALIDAD : String
-PRECIO_BASE : String
-NUMERADA : String
-NUM_ASIENTOS : String
-VENUE_ID : String
+cargarVenuesYLocalidades(archVenues:String, archLocalidades:String) : List<Venue>
+salvarVenues(archivo:String, venues:List<Venue>) : void
+salvarLocalidades(archivo:String, venues:List<Venue>) : void}
PersistenciaEventosJson
-ID_EVENTO : String
-NOMBRE : String
-FECHA : String
-HORA : String
-ESTADO : String
-TIPO_EVENTO : String
-VENUE_ID : String
-TIQUETES : String
+cargarEventos(archivo:String, admin:Administrador, organizadores:
List<Organizador>, venues:List<Venue>) : List<Evento>
+salvarEventos(archivo:String, eventos:List<Evento>) : void
PersistenciaTiquetesJson
-ID_TIQ : String
-TIPO : String
-PRECIO : String
-CARGO_SERVICIO : String
-CARGO_EMISION : String
-ESTADO : String
-PROPIETARIO_LOGIN : String
-EVENTO_ID : String
-LOCALIDAD_ID : String
-NUM_ASIENTO : String
-LOC_NUMERADA : String
-ENTRADAS : String
+cargarTiquetes(archivo:String, usuarios:
List<Usuario>, eventos:List<Evento>) : List<Tiquete>
+salvarTiquetes(archivo:String,
tiquetes:List<Tiquete>) : void
PersistenciaUsuariosJson
-LOGIN : String
-PASSWORD : String
-NOMBRE : String
-SALDO : String
-ID_ADMIN : String
-ID_CLIENTE : String
-ID_ORGANIZADOR : String
-CARGOS_SERVICIO : String
-CARGO_EMISSION : String
-VENUES_APROBADOS : String
-EVENTOS : String
-TIQUETES : String
+cargarUsuarios(archivo:String, admin:Administrador, clientes:List<Cliente>, organizadores:List<Organizador>) : void
+salvarUsuarios(archivo:String, admin:Administrador, clientes:List<Cliente>, organizadores:List<Organizador>) : void
interface IPersistenciaVenues
+cargarVenuesYLocalidades(archVenues:String, archLocalidades:String) : List<Venue>
+salvarVenues(archivo:String, venues:List<Venue>) : void
+salvarLocalidades(archivo:String, venues:List<Venue>) : void
builds
CentralPersistencia
+JSON : String = "JSON"
+PLAIN : String = "PlainText"
+getPersistenciaUsuarios(tipo:String) : IPersistenciaUsuarios
+getPersistenciaEventos(tipo:String) : IPersistenciaEventos
+getPersistenciaVenues(tipo:String) : IPersistenciaVenues
+getPersistenciaTiquetes(tipo:String): IPersistenciaTiquetes
builds
interface IPersistenciaUsuarios
+cargarUsuarios(archivo:String, admin:Administrador, clientes:List<Cliente>, organizadores:List<Organizador>) : void
+salvarUsuarios(archivo:String, admin:Administrador, clientes:List<Cliente>, organizadores:List<Organizador>) : void
Venue
Localidad
interface IPersistenciaEventos
+cargarEventos(archivo:String, admin:Administrador, organizadores:List<Organizador>,
venues:List<Venue>) : List<Evento>
+salvarEventos(archivo:String, eventos:List<Evento>) : void
interface IPersistenciaTiquetes
+cargarTiquetes(archivo:String, usuarios:List<Usuario>, eventos:List<Evento>) : List<Tiquete>
+salvarTiquetes(archivo:String, tiquetes:List<Tiquete>) : void
Evento
abstract Usuario
-login : String
-password: String
-nombre : String
-saldo : double
Tiquete
Administrador
-idAdministrador : String
-venuesAprobados : List<Venue>
-cargoEmision : double
-cargosServicio : Map<TipoEvento,Double>
Cliente
-idCliente : String
-tiquetes : List<Tiquete>
Organizador
-idOrganizador : String
-eventos : List<Evento>
-cortesias : List<Tiquete>
log
entradaLog
-descripcion: String
-fechaHora: LocalDateTime
-tipo: String
+EntradaLog(LocalDateTime, String, String)
+getDescripcion(): String
+getFechaHoral(): LocalDateTime
+getTipo(): String
+hashCode(): int
+toString(): String
entrada
logSistema
entradas: List<EntradaLog>
-consultar(Administrador, String, String): List<EntradaLog>
-getEntradas(): List<EntradaLog>
-limpiar(): void
-reemplazarEntradas(List<EntradaLog>): void}
-registrar(String): EntradaLog